COUNTING SEMAPHORE FOR SHARED RESOURCE MANAGAMENT

Assignment 06

CSE323 Advanced Embedded Systems

Table of Contents

1.0 Executive Objective 2
2.0 System Architecture & Execution Flow 2
2.1 Task Hierarchy & Resource Constraints 2
3.0 Screenshot Verification & Sequential Analysis 2
3.1 Experimental Result Summary 6
4.0 Interpretation & Engineering Trade-offs 7
5.0 Conclusion 7

Figure 1 Baseline System State 3
Figure 2 Initial Resource Allocation 3
Figure 3 Resource Pool Saturation 4
Figure 4 Enforcement of the Resource Limit 5
Figure 5 Dynamic Resource Deallocation 6

1.0 Executive Objective

The primary objective of this report is to empirically demonstrate the application of counting semaphores as a concurrent resource pool manager within a FreeRTOS environment. Unlike a standard mutex (which strictly enforces mutual exclusion for a single resource), a counting semaphore can manage a finite pool of identical resources. This assignment proves that multiple worker tasks can actively compete for access, but the synchronization primitive will categorically prevent the number of simultaneous active users from exceeding the predefined physical limit of the resource pool.

2.0 System Architecture & Execution Flow

The system was designed to simulate a high-contention environment where the demand for a resource exceeds its availability.

2.1 Task Hierarchy & Resource Constraints

1. **The Shared Resource Pool:** The system is constrained to exactly **2** identical shared resources (representing hardware constraints such as limited DMA channels or specific memory banks).
2. **The Synchronization Primitive:** A single counting semaphore is deployed, initialized with a maximum count of 2 and a starting count of 2.
3. **The Worker Tasks (vWorkerTask):** A pool of **5** identical, competing worker tasks are instantiated. Each task executes an infinite loop attempting to:
    - Acquire a resource token via xSemaphoreTake().
    - Assert its active state within the gTaskUsingResource[] tracking array.
    - Increment the gActiveResources counter.
    - Simulate a physical hardware interaction or computational delay using vTaskDelay().
    - Clear its active flags and release the token back to the pool via xSemaphoreGive().

3.0 Screenshot Verification & Sequential Analysis

To capture undeniable proof of the resource pool limitations, the Keil IDE's debugging environment and watch variables were utilized. The watch window specifically tracks gActiveResources, gMaxObservedResources, and the state array gTaskUsingResource[0..4]. By establishing breakpoints inside the critical section, the RTOS scheduling behavior is frozen and verified over time.

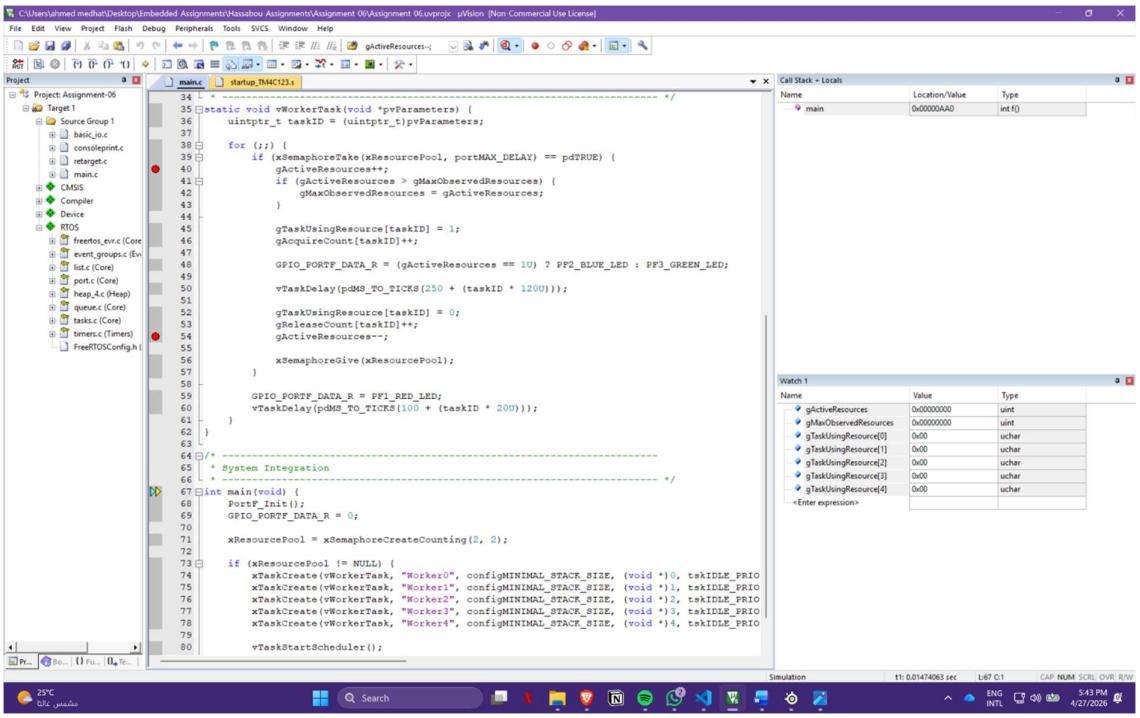

Figure 1 Baseline System State

This capture illustrates the initial debug state prior to the RTOS scheduler taking control. All diagnostic watch variables, including gActiveResources, gMaxObservedResources, and the individual worker tracking array gTaskUsingResource[], remain at zero. This establishes a valid, uncontaminated baseline state.

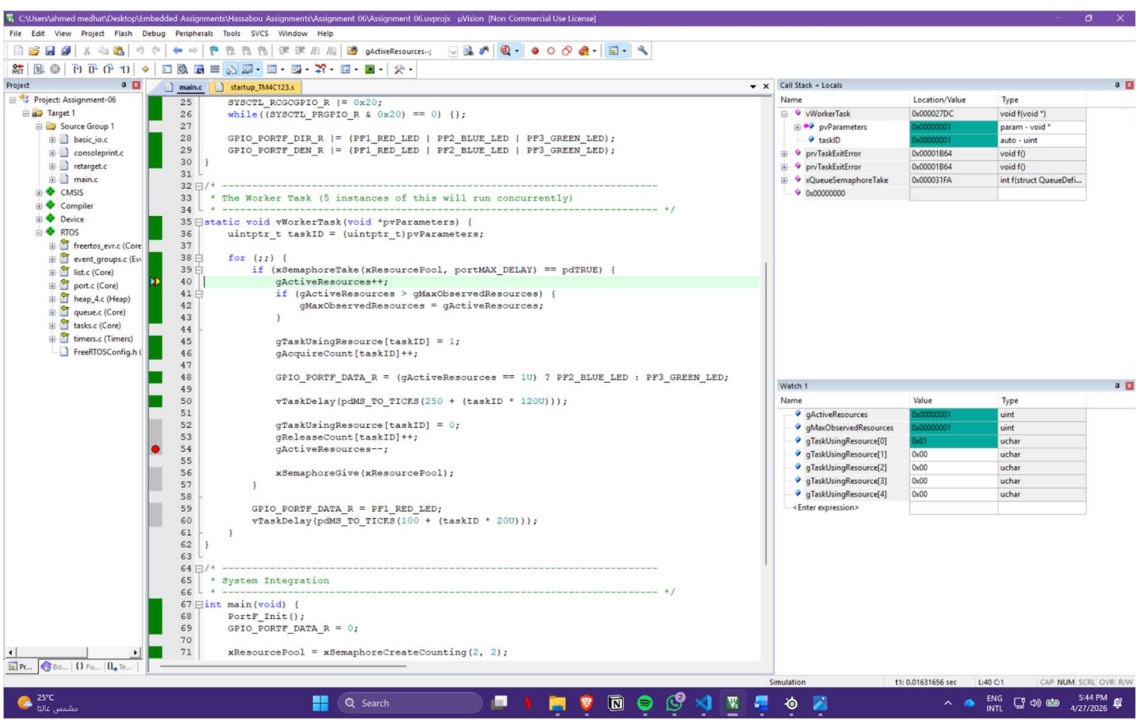

Figure 2 Initial Resource Allocation

The scheduler starts, and the first worker task successfully acquires a token from the counting semaphore.

- gActiveResources = 1
- gMaxObservedResources = 1
- One index within the gTaskUsingResource[] array asserts high (1). This confirms the baseline functionality: a task has entered the critical section and registered its identity.

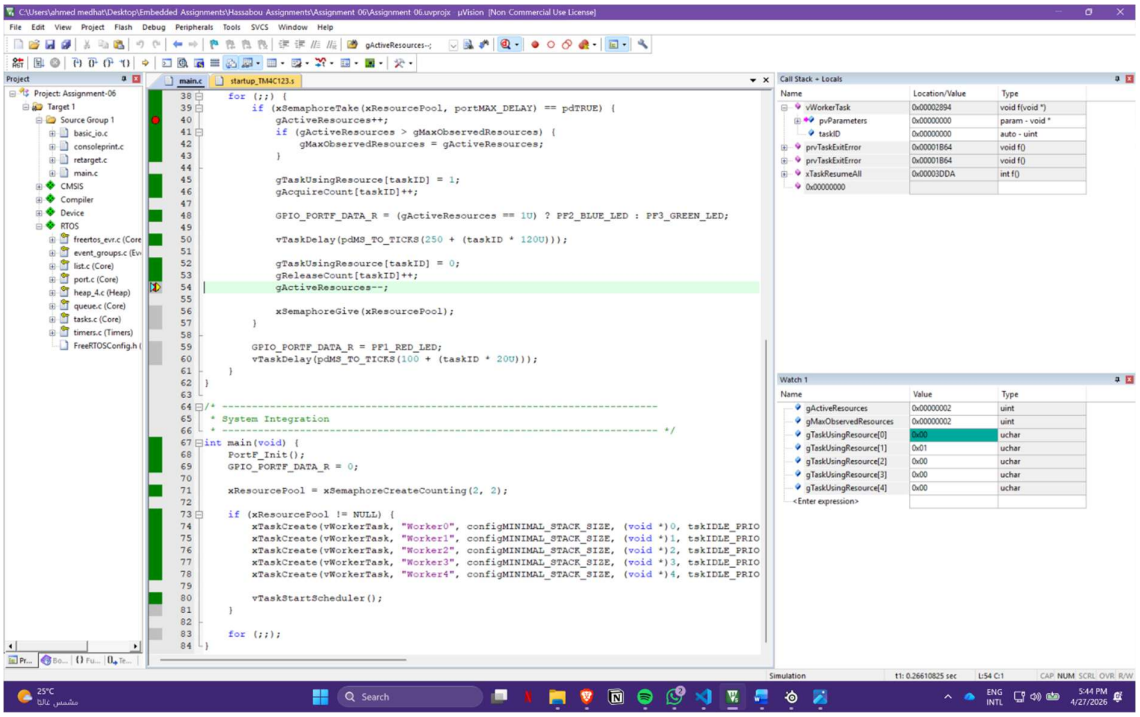

Figure 3 Resource Pool Saturation

Before the first task finishes its simulated delay, a second worker task preempts or is scheduled, successfully acquiring the final available token from the counting semaphore.

- gActiveResources = 2
- gMaxObservedResources = 2
- A second, distinct index within gTaskUsingResource[] asserts high (1). The resource pool is now operating at 100% capacity. Any subsequent calls to xSemaphoreTake() by the remaining three worker tasks must result in a blocked state.

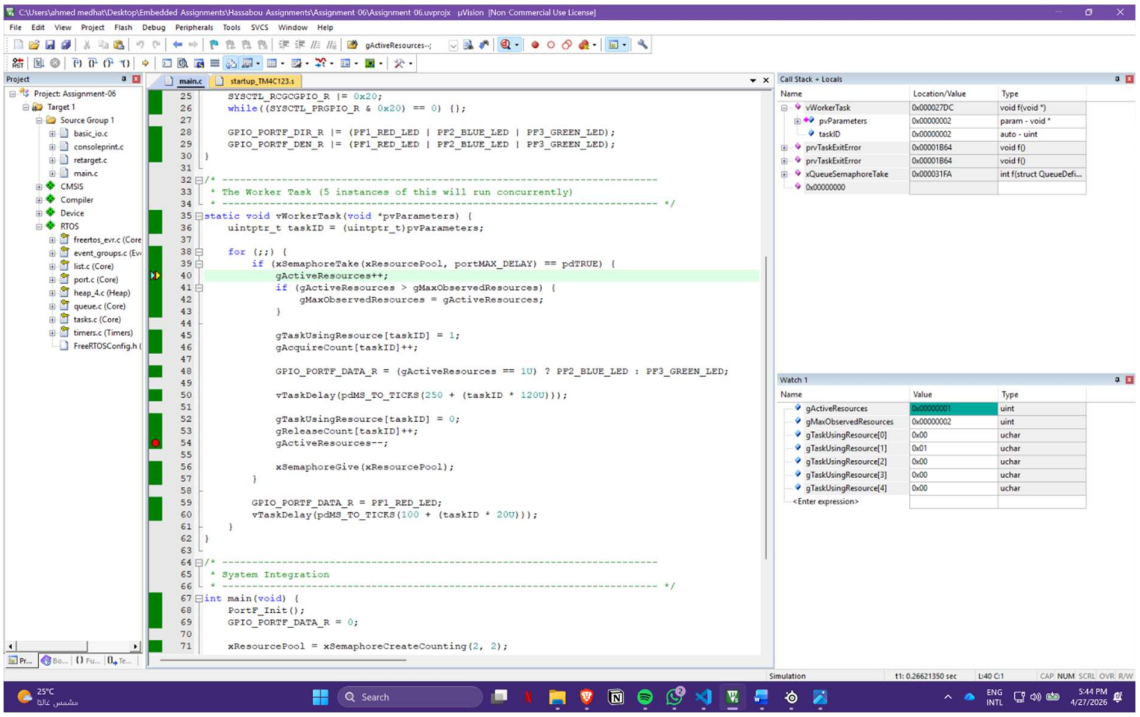

Figure 4 Enforcement of the Resource Limit

Execution continues, but the breakpoints inside the critical section are not triggered by a third task. The watch variables hold steady:

- gActiveResources = 2
- gMaxObservedResources = 2 This is a critical proof state. It demonstrates that while 5 tasks exist and are actively competing, the counting semaphore successfully blocks tasks 3, 4, and 5 from entering the resource section. The hardware limitation is strictly enforced.

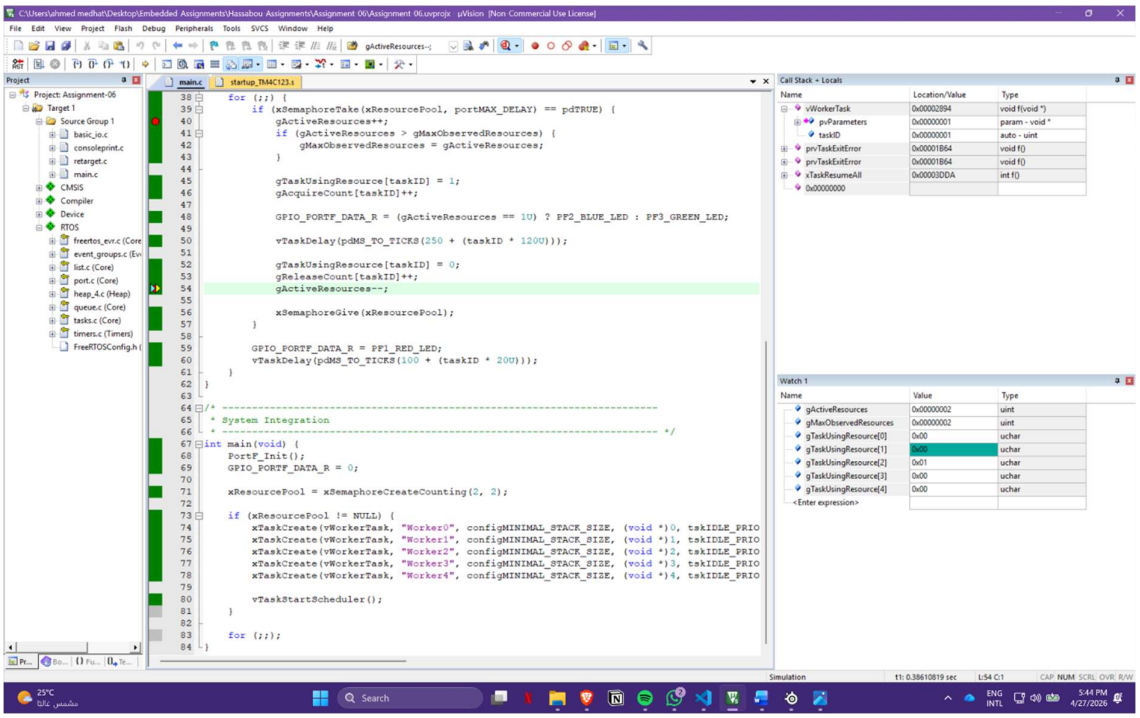

Figure 5 Dynamic Resource Deallocation

A task completes its vTaskDelay() and yields its token via xSemaphoreGive().

- gActiveResources drops back to 1.
- The corresponding task's flag in gTaskUsingResource[] resets to 0.
- gMaxObservedResources remains at 2 (preserving the historical peak). This proves the tasks are not permanently locking the system; resources are being gracefully released back to the RTOS pool.

3.1 Experimental Result Summary

The captured execution confirms the expected mathematical behavior of the counting semaphore resource pool:

1. Worker tasks dynamically enter and leave the resource section over time.
2. The identity of the active workers (gTaskUsingResource[]) safely rotates among the 5 competitors.
3. The gActiveResources counter rises and falls sequentially.
4. The critical boundary condition is maintained: gMaxObservedResources reaches exactly 2 and never exceeds it, empirically proving the semaphore's limitation enforcement.

4.0 Interpretation & Engineering Trade-offs

This experiment highlights a fundamental architectural decision in embedded systems. While a Mutex is ideal for protecting a singular, vulnerable data structure (like a specific UART peripheral), it creates massive bottlenecks if applied to parallel capabilities. By using a counting semaphore initialized to the exact number of physical resources (e.g., a dual-channel ADC or two independent SPI buses), an engineer can maximize throughput. The RTOS natively handles the queueing and blocking of excess tasks, eliminating the need for complex, manual software-level tracking arrays.

5.0 Conclusion

The assignment objectives were successfully achieved. The implemented FreeRTOS application demonstrates that a counting semaphore can securely manage a limited pool of two shared resources among five competing worker tasks. The debugger evidence definitively proves that while the active task identities continuously alternate, the total number of simultaneous resource users never exceeds the established limit of 2.